

Informe técnico — Ecosistema de Microservicios Auth + Catálogo

Asignatura: Desarrollo Web — Semana 14

Proyecto: S14 — Microservicios Django (Auth + Catalog)

Stack: Django + DRF · PostgreSQL · Docker Compose · Kubernetes · JWT · Postman

Estudiante:

Alarcón Mendoza Estiven Rodrigo

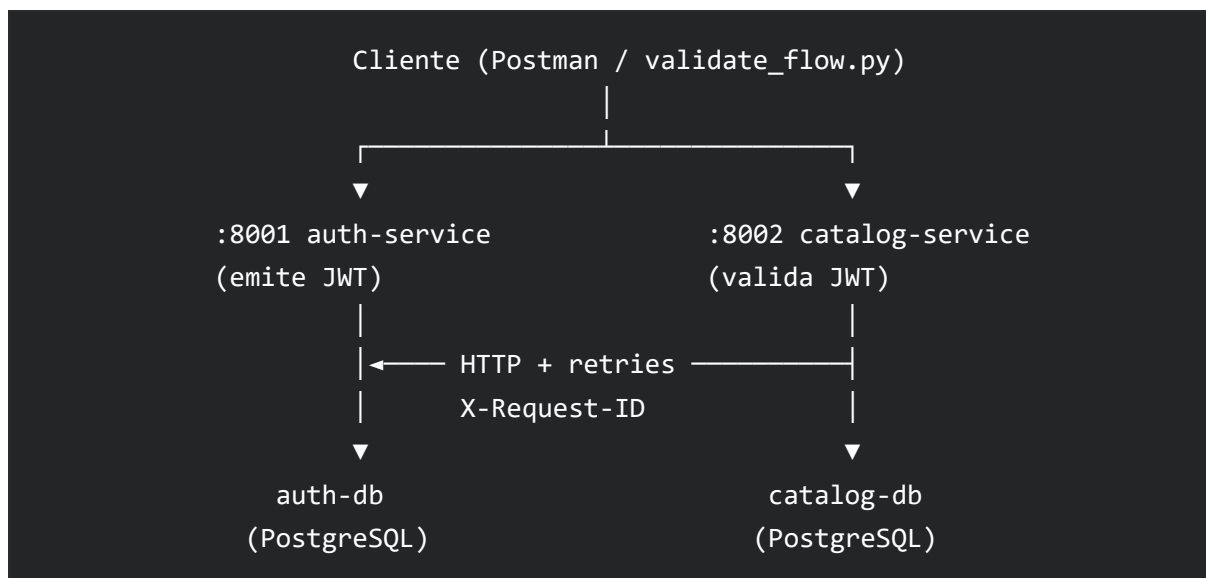
Cruz Cruz Alexander Jhon

1. Objetivo

Implementar un ecosistema de microservicios con:

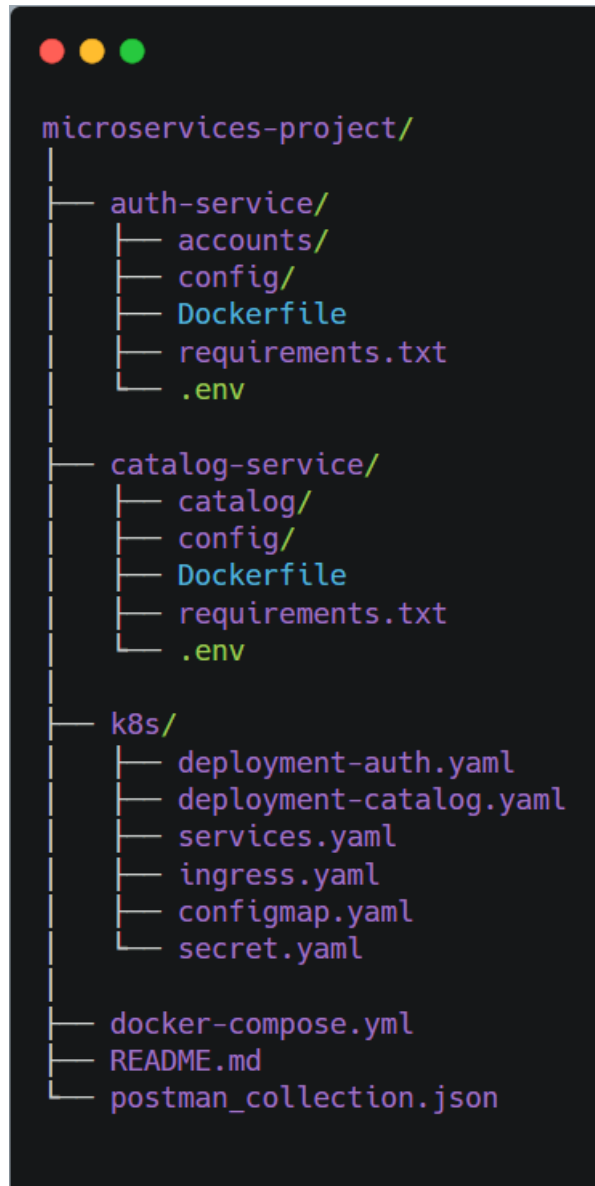
1. Arquitectura e aislamiento (dos servicios Django, DB separada, env segregadas).
2. Contenedorización y orquestación (Dockerfile multi-stage, Compose, manifiestos K8s).
3. Comunicación y seguridad (JWT, cliente inter-servicio, **X-Request-ID**).
4. Monitoreo, health y escalado (**/health/**, **/live/**, probes, métricas, scale).
5. Consumo y validación (script cliente, Postman, documentación).

2. Arquitectura general



Cada servicio es un proyecto Django independiente: código, imagen Docker, variables de entorno y base de datos propios.

3. Estructura de carpetas y archivos importantes



Mapa rápido: archivo → responsabilidad

Archivo / carpeta	Por qué es importante
<code>docker-compose.yml</code>	Levanta 2 apps + 2 Postgres aislados
<code>auth-service/accounts/views.py</code>	Emisión de JWT y health

<code>catalog-service/catalog/middleware.py</code>	Validación Bearer en cada request
<code>catalog-service/catalog/utils/client.py</code>	Comunicación inter-servicio resiliente
<code>*/Dockerfile</code>	Imagen optimizada y usuario no-root
<code>k8s/deployment-*.yaml</code>	Réplicas, probes, ConfigMap/Secret
<code>clients/validate_flow.py</code>	Demuestra el flujo completo
<code>postman_collection.json</code>	Pruebas manuales en Postman

4. Código dentro del proyecto

4.1 Emisión de JWT — `auth-service`

Tras autenticar al usuario, se firma un token HS256 con `JWT_SECRET` compartido (solo el secreto; las DB siguen aisladas).

```
# auth-service/accounts/views.py (LoginView.post)
user = authenticate(
    username=serializer.validated_data["username"],
    password=serializer.validated_data["password"],
)
if user is None:
    return Response({"error": "Credenciales inválidas"}, status=401)

payload = {
    "user_id": user.id,
    "username": user.username,
    "exp": datetime.datetime.utcnow()
        + datetime.timedelta(hours=settings.JWT_EXPIRATION_HOURS),
    "iat": datetime.datetime.utcnow(),
}
token = jwt.encode(payload, settings.JWT_SECRET,
algorithm=settings.JWT_ALGORITHM)
return Response({"token": token, "token_type": "Bearer", ...})
```

Endpoints auth:

Método	Ruta	Descripción
POST	<code>/api/auth/register/</code>	Crear usuario

POST	/api/auth/login/	Obtener JWT
GET	/api/auth/me/	Validar token
GET	/health/	Readiness (DB)
GET	/live/	Liveness
GET	/metrics/	Métricas

4.2 Validación JWT en catálogo — middleware

Todas las rutas (salvo health/live/metrics/admin) exigen `Authorization: Bearer <token>`.

```
# catalog-service/catalog/middleware.py
class JWTAuthMiddleware(MiddlewareMixin):
    PUBLIC_PREFIXES = ("/admin/", "/health/", "/live/", "/metrics/")

    def process_request(self, request):
        if any(request.path.startswith(p) for p in self.PUBLIC_PREFIXES):
            return None

        auth_header = request.headers.get("Authorization")
        if not auth_header or not auth_header.startswith("Bearer "):
            return JsonResponse({"error": "Token missing"}, status=401)

        token = auth_header.split(" ", 1)[1]
        try:
            payload = jwt.decode(
                token, settings.JWT_SECRET,
                algorithms=[settings.JWT_ALGORITHM]
            )
            request.user_id = payload.get("user_id")
        except jwt.ExpiredSignatureError:
            return JsonResponse({"error": "Token expired"}, status=401)
        except jwt.InvalidTokenError:
            return JsonResponse({"error": "Invalid token"}, status=401)
```

Esto explica el mensaje `{"error": "Token missing"}` al abrir el catálogo sin token en el navegador.

4.3 Cliente inter-servicio — retries, timeout, tracing

```
# catalog-service/catalog/utils/client.py
_retry = Retry(
    total=3,
    backoff_factor=0.5,
    status_forcelist=[500, 502, 503, 504],
)
_session.mount("http://", HTTPAdapter(max_retries=_retry))

def call_auth_service(endpoint, ..., timeout=3.0, request_id=None):
    base = os.getenv("AUTH_SERVICE_URL", "http://auth-service:8000")
    headers.setdefault("X-Request-ID", request_id or uuid.uuid4().hex)
    try:
        return _session.request(method, url, headers=headers,
                                timeout=timeout)
    except requests.RequestException as exc:
        raise AuthServiceError(str(exc), status_code=503) from exc
```

Usado en `/health/` (dependencia auth) y en `/api/catalog/verify-user/`.

4.4 Health con chequeo de DB y dependencia

```
# catalog-service/catalog/views.py (health)
# 1) DB local
with connection.cursor() as cursor:
    cursor.execute("SELECT 1")
checks["db"] = "connected"

# 2) Dependencia auth-service
resp = call_auth_service("/health/", timeout=2.0, request_id=request_id)
checks["auth_service"] = {"status_code": resp.status_code, "ok":
resp.status_code == 200}
# Si auth falla → HTTP 503 (readiness)
```

Separación de probes:

Endpoint	Uso en K8s	Qué verifica
----------	------------	--------------

<code>/live/</code>	<code>livenessProbe</code>	Proceso vivo
<code>/health/</code>	<code>readinessProbe</code>	DB + dependencias

Así, si auth cae, catalog deja de recibir tráfico (readiness) sin reiniciarse en bucle (liveness).

4.5 Modelo de dominio del catálogo

```
# catalog-service/catalog/models.py
class Product(models.Model):
    name = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
    category = models.CharField(max_length=100, blank=True, default="")
    is_active = models.BooleanField(default=True)
```

DB propia (`catalogdb`); no comparte tablas con auth.

4.6 Aislamiento de bases en settings

```
# auth-service/config/settings.py (patrón equivalente en catalog)
_database_url = os.getenv("DATABASE_URL")
if _database_url:
    _parsed = urlparse(_database_url)
    DATABASES = {
        "default": {
            "ENGINE": "django.db.backends.postgresql",
            "NAME": _parsed.path.lstrip("/"),
            "USER": _parsed.username,
            "PASSWORD": _parsed.password,
            "HOST": _parsed.hostname,
            "PORT": _parsed.port or 5432,
        }
    }
```

En Compose:

- Auth → postgres://authuser:authpass@auth-db:5432/authdb
- Catalog → postgres://cataloguser:catalogpass@catalog-db:5432/catalogdb

4.7 Docker multi-stage + non-root

```
# auth-service/Dockerfile (igual patrón en catalog)
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.11-slim
RUN groupadd -r appgroup && useradd -r -g appgroup -u 10001 appuser
COPY --from=builder /install /usr/local
COPY . .
USER appuser
EXPOSE 8000
ENTRYPOINT ["python", "entrypoint.py"]
```

El stage `builder` instala dependencias; el runtime solo copia artefactos y corre como `appuser` (UID 10001).

4.8 Docker Compose — servicios aislados

```
# docker-compose.yml (extracto)
auth-db:
  image: postgres:15-alpine
  environment:
    POSTGRES_DB: authdb
    POSTGRES_USER: authuser
    POSTGRES_PASSWORD: authpass

catalog-db:
  image: postgres:15-alpine
  environment:
    POSTGRES_DB: catalogdb
    POSTGRES_USER: cataloguser
    POSTGRES_PASSWORD: catalogpass
```

```

auth-service:
  ports: ["8001:8000"]
  environment:
    DATABASE_URL: postgres://authuser:authpass@auth-db:5432/authdb
    JWT_SECRET: dev-secret-change-in-prod

catalog-service:
  ports: ["8002:8000"]
  environment:
    DATABASE_URL: postgres://cataloguser:catalogpass@catalog-
db:5432/catalogdb
    AUTH_SERVICE_URL: http://auth-service:8000
    JWT_SECRET: dev-secret-change-in-prod

```

4.9 Kubernetes — probes y secretos

```

# k8s/deployment-auth.yaml (extracto)
replicas: 2
env:
  - name: DATABASE_URL
    valueFrom:
      secretKeyRef:
        name: auth-db-secret
        key: url
  - name: JWT_SECRET
    valueFrom:
      secretKeyRef:
        name: auth-db-secret
        key: jwt
readinessProbe:
  httpGet: { path: /health/, port: 8000 }
livenessProbe:
  httpGet: { path: /live/, port: 8000 }

```

Escalado validado:

```
kubectl -n microservices scale deployment/catalog-deploy --replicas=3
```

Manifiestos clave en


```
k8s/: namespace, secrets, configmaps, postgres-*, deployment-*, ingress.
```

4.10 Tracing con X-Request-ID

```
# Ambos servicios – RequestIDMiddleware
def process_request(self, request):
    request_id = request.META.get("HTTP_X_REQUEST_ID") or
    uuid.uuid4().hex
    request.request_id = request_id

def process_response(self, request, response):
    response["X-Request-ID"] = getattr(request, "request_id",
    uuid.uuid4().hex)
    return response
```

El cliente y el catalog propagan el mismo ID hacia auth para correlacionar logs.

5. Flujo de validación (consumo)

Script

```
docker compose up --build -d
python clients/validate_flow.py
python clients/validate_flow.py --burst # 429
docker compose stop auth-service
python clients/validate_flow.py --simulate-503 # 503
```

Postman

Importar `postman_collection.json` → Login (guarda `token`) → List Products → casos 401.

Manejo de errores demostrado

Código	Caso	Dónde
--------	------	-------

401	Sin token / token inválido	Middleware catalog
429	Throttle login (30/min)	BurstAnonThrottle en auth
503	Auth caído o DB caída	/health/ y call_auth_service

6. Resumen

1. **Dos proyectos Django** en lugar de un monólito modular → aislamiento real de despliegue y fallo.
2. **JWT HS256 compartido** → catalog valida sin llamar a auth en cada request de productos; la llamada inter-servicio se usa en health y verify-user.
3. **/live/ vs /health/** → evita reinicios en cascada en Kubernetes.
4. **Secrets / ConfigMaps** → credenciales fuera del código de Deployment.
5. **Retries + timeout** → tolerancia a fallos transitorios entre servicios.
6. **Métricas en texto Prometheus** → suficiente para la práctica; en producción se usaría un exporter dedicado.

7. Cómo ejecutar y detener

```
# Ejecutar
docker compose up --build -d

# Estado
docker compose ps
# Auth:      http://localhost:8001/health/
# Catálogo: http://localhost:8002/health/

# Detener (conserva datos)
docker compose stop

# Detener y quitar contenedores
docker compose down

# Detener y borrar volúmenes (datos DB)
docker compose down -v
```

Usuario demo: demo / demo1234

Nota: no hay UI HTML en `/`. Los servicios son API JSON; la interacción se hace con Postman, el script cliente o endpoints `/health/` y `/metrics/` en el navegador.

Preguntas

- **¿Cuándo conviene usar un monolito?**

Conviene en aplicaciones pequeñas o medianas, con pocos desarrolladores y requisitos simples, ya que facilita el desarrollo, despliegue y mantenimiento.

- **¿Cuándo convienen los microservicios?**

Son adecuados para sistemas grandes o complejos que requieren escalabilidad, despliegues independientes y equipos de desarrollo trabajando en distintos módulos.

- **¿Qué ocurre si no existe readinessProbe?**

Kubernetes puede enviar tráfico a un contenedor que aún no está listo, provocando errores de conexión, respuestas 500/503 y afectando la disponibilidad del servicio.

- **¿Cómo Kubernetes realiza escalado sin *downtime*?**

Kubernetes crea nuevos Pods y solo dirige tráfico hacia ellos cuando están listos. Luego reemplaza gradualmente los Pods antiguos mediante actualizaciones progresivas (*Rolling Update*), evitando interrupciones del servicio.

Resultado final

GitHub: [Ichimar465/S14-Microservicios-Django-y-Docker](https://github.com/Ichimar465/S14-Microservicios-Django-y-Docker)

```
PS D:\IX semestre\Desarrollo Web\Semana 14\S14> docker compose up --build -d
[+] up 6/6
✓ Image s14-auth-service      Built      1.9s
✓ Image s14-catalog-service   Built      1.9s
✓ Container s14-catalog-db-1   Healthy    2.6s
✓ Container s14-auth-db-1      Healthy    1.9s
✓ Container s14-auth-service-1 Healthy    7.6s
✓ Container s14-catalog-service-1 Started     7.2s
PS D:\IX semestre\Desarrollo Web\Semana 14\S14> docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS
s14-auth-db-1        postgres:15-alpine                 "docker-entrypoint.s..."           auth-db     3 minutes ago  Up 3 minutes
s14-auth-service-1   s14-auth-service                   "python entrypoint.py"               auth-service 10 seconds ago Up 8 seconds
s14-catalog-db-1     postgres:15-alpine                 "docker-entrypoint.s..."           catalog-db   3 minutes ago  Up 3 minutes
s14-catalog-service-1 s14-catalog-service               "python entrypoint.py"               catalog-service 9 seconds ago  Up 2 seconds
```

```
PS D:\IX semestre\Desarrollo Web\Semana 14\S14> python clients/validate_flow.py
(usa --simulate-503 tras detener auth-service para forzar 503)
```

=== 7. Métricas ===

```
[OK] auth /metrics/ -> HTTP 200
```

```
      X-Request-ID: 9099a6b463aa40f5964674730af01c24
```

```
[OK] catalog /metrics/ -> HTTP 200
```

```
      X-Request-ID: 80e90a0ed5644b3fb3b607de4653b3bc
```

=== Resumen ===

Flujo validado correctamente.

✓ S14 Ecosistema Microservicios Auth+Catalog

✓ 1. Health

GET Auth Health

GET Auth Live

GET Catalog Health

GET Catalog Live

✓ 2. Auth

POST Register

POST Login

GET Me (validar token)

POST Login inválido → 401

✓ 3. Catalog

GET List Products

POST Create Product

GET Get Product by ID

GET Verify User (inter-servicio)

GET Sin token → 401

GET Token inválido → 401

✓ 4. Errores 429 / 503 / Metrics

GET Auth Metrics

GET Catalog Metrics

POST Burst Login (429) — Runner x50

GET 503 — detener auth y llamar health/verify

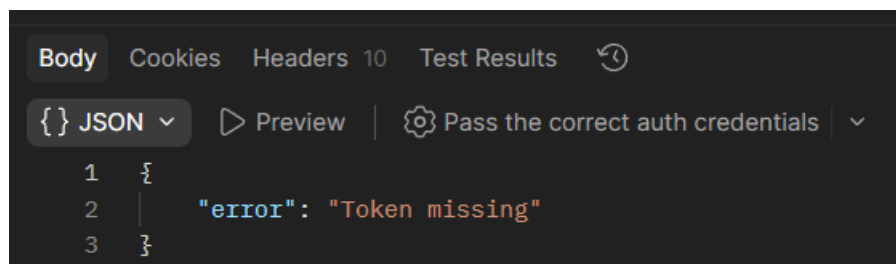
Body Cookies Headers 12 Test Results ↺

{ } JSON ▾

▶ Preview

🖼 Visualize ▾

```
1 {
2   "service": "auth-service",
3   "status": "healthy",
4   "db": "connected"
5 }
```



```
Body Cookies Headers 10 Test Results ↺
{{}} JSON ▾ ▶ Preview | ⚙ Pass the correct auth credentials ▾
1 {
2   "error": "Invalid token"
3 }
```

```
Body Cookies Headers 12 Test Results ↺ 200 OK
Raw ▾ ▶ Preview 🖼 Visualize ▾
1 # HELP auth_requests_total Contadores de requests del auth-service
2 # TYPE auth_requests_total counter
3 auth_requests_total{event="http_200"} 155
4 auth_requests_total{event="http_201"} 1
5 auth_requests_total{event="http_404"} 1
6 auth_requests_total{event="http_405"} 1
7 auth_requests_total{event="http_total"} 158
8 auth_requests_total{event="login_requests"} 2
9 auth_requests_total{event="login_success"} 2
10 auth_requests_total{event="register_requests"} 1
11 # HELP auth_uptime_seconds Segundos desde el arranque del proceso
12 # TYPE auth_uptime_seconds gauge
13 auth_uptime_seconds 994
```

```
Body Cookies Headers 12 Test Results ↺ 200 OK
Raw ▾ ▶ Preview 🖼 Visualize ▾
1 # HELP catalog_requests_total Contadores del catalog-service
2 # TYPE catalog_requests_total counter
3 catalog_requests_total{event="auth_missing"} 3
4 catalog_requests_total{event="auth_ok"} 2
5 catalog_requests_total{event="http_200"} 50
6 catalog_requests_total{event="http_total"} 50
7 catalog_requests_total{event="products_list"} 2
8 # HELP catalog_uptime_seconds Segundos desde el arranque
9 # TYPE catalog_uptime_seconds gauge
10 catalog_uptime_seconds 1008
11
```